

Resumo explicado — RNA para Classificação e Regressão

Student Habits vs Academic Performance

Este material explica, etapa por etapa, o que o notebook faz **e por que** cada decisão foi tomada. A ideia é servir tanto para estudar quanto para apresentar.

1. Ideia central do trabalho

O notebook usa **Redes Neurais Artificiais (RNA)** para resolver **dois problemas diferentes** a partir do mesmo conjunto de dados de hábitos de estudantes:

Tarefa	O que prevê	Tipo de saída	Alvo
Classificação	Categoria de desempenho	Texto: <code>Baixo</code> , <code>Medio</code> ou <code>Alto</code>	<code>score_class</code>
Regressão	Nota numérica exata	Número: ex. <code>78.6</code>	<code>exam_score</code>

Por que separar os dois? Porque são problemas conceitualmente distintos:

- A **classificação** responde "*em qual faixa o aluno está?*" — útil quando você só precisa categorizar (ex.: sinalizar quem está em risco).
- A **regressão** responde "*qual nota exata o aluno tira?*" — útil quando você precisa do valor em pontos.

Colocar os dois lado a lado no mesmo notebook tem valor didático: mostra que a **mesma técnica (RNA)** se adapta a objetivos diferentes só trocando a "cabeça" do modelo e as métricas de avaliação.

2. O dataset

Arquivo: `student_habits_performance.csv` — **1000 registros** e **16 colunas**.

As colunas descrevem hábitos e características dos alunos:

- **Numéricas:** `age`, `study_hours_per_day`, `social_media_hours`, `netflix_hours`, `attendance_percentage`, `sleep_hours`, `exercise_frequency`, `mental_health_rating`
- **Catégoricas:** `gender`, `part_time_job`, `diet_quality`, `parental_education_level`, `internet_quality`, `extracurricular_participation`
- **Identificador:** `student_id`
- **Alvo numérico:** `exam_score`

Valores ausentes: só uma coluna tinha problema — `parental_education_level`, com **91 valores faltando**. Esse detalhe é importante porque justifica uma das etapas do pré-processamento (imputação, explicada mais abaixo).

Por que essa inspeção inicial importa? Antes de modelar, você precisa saber o que tem em mãos: quantos dados, quais tipos, onde há buracos. É o que define como tratar tudo depois.

3. Imports e configuração

O notebook importa o `scikit-learn` (pandas/numpy para dados, matplotlib para gráficos, joblib para salvar modelos). Dois pontos merecem atenção:

- `RANDOM_STATE = 42` — uma "semente" fixa de aleatoriedade. Garante que toda vez que o notebook roda, a divisão dos dados e o treino da rede dão **o mesmo resultado**. Isso é essencial para reprodutibilidade: sem isso, cada execução daria números ligeiramente diferentes e ninguém conseguiria conferir seu trabalho.
- `DummyClassifier` / `DummyRegressor` — modelos "bobos" usados como **baseline** (linha de base). Servem de referência mínima para provar que a RNA realmente aprendeu algo, e não só chutou. Explico melhor nas partes A e B.

4. Criação da classe (`score_class`)

A classificação precisa de um alvo categórico, mas o dataset só tinha a nota numérica. Então a classe foi **criada a partir da nota**:

- `Baixo` : nota < 60
- `Medio` : nota ≥ 60 e < 80
- `Alto` : nota ≥ 80

Resultado da distribuição:

Classe	Quantidade
Baixo	280
Medio	443
Alto	277

Por que esses cortes? Seguem uma lógica escolar comum (reprovado / mediano / bom). E repare que a distribuição é **desbalanceada**: a classe `Medio` tem quase o dobro das outras. Isso vai justificar duas escolhas mais à frente: a **estratificação** na divisão treino/teste e o uso de **métricas "macro"** na avaliação.

5. Separação de entradas (X) e alvos (y)

```
X      = tudo, menos student_id, exam_score e score_class
y_class = score_class    (alvo da classificação)
y_reg   = exam_score     (alvo da regressão)
```

Por que remover `student_id`? É só um identificador — não carrega informação preditiva. Se entrasse no modelo, viraria ruído (ou pior, o modelo poderia "decorar" alunos em vez de aprender padrões).

Por que remover os dois alvos do X? Porque seria "colar na prova": `score_class` é derivada de `exam_score`, então deixar qualquer um dos dois na entrada entregaria a resposta de graça e o modelo pareceria perfeito sem ter aprendido nada (vazamento de dados / *data leakage*).

Em seguida, o código separa automaticamente as colunas em **numéricas** e **categóricas**, porque cada tipo precisa de um tratamento diferente.

6. Divisão treino / teste

```
80% treino (800 registros) | 20% teste (200 registros)
```

Pontos-chave da escolha:

- **`stratify=y_class`** — a divisão mantém a **mesma proporção de classes** no treino e no teste. Como os dados são desbalanceados, sem isso o teste poderia, por azar, ficar com poucos alunos **Alto**, distorcendo a avaliação.
- **Mesmo conjunto de teste para as duas tarefas** — feito apenas para organizar (os mesmos 200 alunos são avaliados nas duas pontas). As métricas, porém, **não são comparáveis entre si** porque medem coisas diferentes.

Por que separar treino e teste? O modelo aprende no treino e é avaliado no teste — dados que ele **nunca viu**. É a única forma honesta de medir se ele generaliza ou só decorou.

7. Pré-processamento (o pipeline)

Esta é uma das partes mais importantes. O código monta um **pré-processador** que trata cada tipo de coluna de um jeito:

Colunas numéricas: 1. `SimpleImputer(strategy='median')` — preenche valores faltantes com a mediana. 2. `StandardScaler()` — **padroniza** (média 0, desvio 1).

Colunas categóricas: 1. `SimpleImputer(strategy='most_frequent')` — preenche faltantes com a **moda** (valor mais comum). 2. `OneHotEncoder` — transforma categorias em colunas 0/1.

Por que cada escolha?

- **Mediana em vez de média** nos numéricos: a mediana é robusta a valores extremos (*outliers*). Se um aluno tem um valor muito atípico, ele não puxa a imputação.
- **Padronização (`StandardScaler`)**: redes neurais são **muito sensíveis à escala** dos dados. Sem padronizar, uma variável que vai de 0 a 100 (ex.: `attendance_percentage`) dominaria outra que vai de 0 a 10, e o treino ficaria instável/lento. Padronizar coloca todas no mesmo "patamar".
- **Moda nas categóricas**: faz sentido preencher um campo de texto faltante com a categoria mais frequente (resolve os 91 buracos de `parental_education_level`).
- **One-Hot Encoding**: o modelo só entende números. O One-Hot transforma `internet_quality = "Alta"` em colunas binárias, **sem inventar uma ordem falsa** entre as categorias (diferente de numerar 1, 2, 3, que sugeriria que uma é "maior" que a outra).
- `handle_unknown='ignore'` : se aparecer uma categoria nova no teste que não existia no treino, o modelo não quebra.

Por que usar `Pipeline` + `ColumnTransformer` ? Para encapsular pré-processamento e modelo num objeto só. Isso garante que **as mesmas transformações** sejam aplicadas no treino e no teste, evita vazamento de dados (o scaler "aprende" só com o treino) e deixa o código limpo e reaproveitável.

8. Gráficos exploratórios

Antes de modelar, o notebook gera:

- **Distribuição de `score_class`** (gráfico de barras) — mostra o desbalanceamento.
- **Distribuição de `exam_score`** (histograma) — mostra como as notas se espalham.
- **Matriz de correlação** das variáveis numéricas — mostra quais variáveis andam juntas (ex.: horas de estudo provavelmente se correlacionam positivamente com a nota).

Por que fazer isso? Entender os dados antes de modelar ajuda a interpretar os resultados depois e a desconfiar de números estranhos.

PARTE A — RNA para Classificação

9. O modelo classificador

Usa um `MLPClassifier` (Multi-Layer Perceptron — uma rede neural feedforward) com:

Parâmetro	Valor	Por quê
<code>hidden_layer_sizes=(64, 32)</code>	2 camadas ocultas (64 e 32 neurônios)	Capacidade suficiente para o padrão, sem exagerar (evita overfitting). O funil 64 → 32 vai refinando a representação.
<code>activation='relu'</code>	ReLU	Ativação padrão moderna; treina rápido e evita o problema do gradiente sumir.
<code>solver='adam'</code>	Adam	Otimizador eficiente e robusto, ótimo padrão para a maioria dos casos.
<code>alpha=0.001</code>	Regularização L2	Penaliza pesos muito grandes → reduz overfitting.
<code>learning_rate_init=0.001</code>	Taxa de aprendizado	Passo de ajuste a cada época; valor conservador e estável.
<code>max_iter=1500</code>	Máx. de épocas	Teto de iterações para o treino convergir.
<code>n_iter_no_change=30</code>	Paciência	Para de treinar se não melhorar por 30 épocas seguidas.

O **baseline**: um `DummyClassifier(strategy='most_frequent')`, que **sempre chuta a classe mais comum** (**Medio**). Serve para responder: "a RNA é melhor do que simplesmente chutar a classe majoritária?"

10. Resultados da classificação

Modelo	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Baseline (classe majoritária)	0,445	—	0,333	0,205
RNA (MLPClassifier)	0,830	0,836	0,832	0,834

Relatório por classe (RNA):

Classe	Precision	Recall	F1	Suporte
Baixo	0,85	0,84	0,85	56
Medio	0,80	0,82	0,81	89
Alto	0,85	0,84	0,84	55

Como ler isso: - A RNA acertou **83% das classificações** contra apenas **44,5%** do chute simples → a rede **aprendeu de verdade**. - As métricas são equilibradas entre as três classes (todas ~0,8), o que é ótimo: o modelo não está bom só na classe majoritária.

Por que essas métricas e não só a acurácia? - **Accuracy** sozinha engana em dados desbalanceados (o baseline tira 44,5% só chutando **Medio**). - **Precision** = das vezes que previu uma classe, quantas estavam certas. - **Recall** = dos alunos que realmente eram daquela classe, quantos foram pegos. - **F1** =

equilíbrio entre precision e recall. - **"Macro"** = média simples entre as classes, dando **peso igual** a cada uma — justo quando há desbalanceamento.

11. Gráficos da classificação

- **Curva de perda (loss):** mostra a perda caindo ao longo das épocas. Se desce e estabiliza, o treino convergiu bem.
- **Matriz de confusão:** mostra onde o modelo acerta e erra. Os erros tendem a ser entre classes vizinhas (confundir **Medio** com **Baixo** ou **Alto**), o que é esperado e "menos grave".
- **Comparação de métricas (barras):** RNA vs baseline lado a lado, evidenciando o ganho.

PARTE B — RNA para Regressão

12. O modelo regressor

Usa um **MLPRegressor** com arquitetura igual à da classificação **(64, 32)**, mas com algumas diferenças importantes:

Diferença	Valor	Por quê
<code>early_stopping=True</code>	Ligado	Separa parte do treino para validação e para sozinho quando a validação para de melhorar → combate overfitting.
<code>max_iter=2000</code>	Mais épocas	Regressão pode precisar de mais ajuste fino.
<code>batch_size=32</code>	Mini-lotes	Atualiza os pesos em lotes de 32 → treino mais estável.
<code>np.clip(pred, 0, 100)</code>	Pós-processamento	Trava as previsões entre 0 e 100 , porque nota não pode ser negativa nem passar de 100.

O baseline: um **DummyRegressor(strategy='mean')**, que **sempre chuta a nota média** do treino. Responde: "a RNA prevê melhor do que apenas dizer a média para todo mundo?"

13. Resultados da regressão

Modelo	MAE	MSE	RMSE	R ²
Baseline (média)	13,48	270,20	16,44	~0,00
RNA (MLPRegressor)	4,02	26,65	5,16	0,901

Como ler isso: - **MAE = 4,02** → em média, a previsão erra a nota real por **~4 pontos**. Muito bom. - **RMSE = 5,16** → erro típico penalizando mais os erros grandes; também baixo. - **R² = 0,901** → o modelo explica **~90% da variação** das notas. Excelente. - O baseline tem **R² ≈ 0** (não explica nada, por definição) e MAE de ~13,5 pontos. A RNA reduziu o erro em mais de **3x**.

Por que essas métricas? - **MAE** (erro absoluto médio): intuitivo, na mesma unidade da nota ("erro de X pontos"). - **MSE / RMSE**: penalizam mais os erros grandes; o RMSE volta para a escala da nota. - **R²**: mede quanto da variação o modelo captura (0 = inútil, 1 = perfeito).

14. Gráficos da regressão

- **Curva de perda**: acompanha a convergência do treino.
- **Real vs Previsto (dispersão)**: quanto mais os pontos colam na linha diagonal tracejada, melhor. Aqui ficam bem próximos.
- **Resíduos (erro vs previsto)**: os erros devem se espalhar **aleatoriamente em torno do zero**, sem padrão. Se houver padrão (ex.: erra mais nas notas altas), indica que o modelo está perdendo algo.

15. Salvamento e metadados

No final, o notebook:

- Salva os **dois modelos treinados** (`.joblib`) → permite reusar sem retreinar.
- Salva um **CSV de previsões** com nota real, nota prevista e o erro de cada aluno do teste → facilita auditar onde o modelo erra.
- Gera um `metadata_modelos.json` documentando dataset, tamanhos, preditores, arquiteturas e métricas → boa prática de rastreabilidade.

16. Conclusão e pontos para a apresentação

O que o trabalho mostra: 1. A **mesma técnica (RNA)** resolve dois tipos de problema só ajustando a saída e as métricas — `MLPClassifier` para categorias, `MLPRegressor` para números. 2. Os **dois modelos superam com folga seus baselines**, provando que aprenderam padrões reais (83% vs 44,5% na classificação; R² 0,90 vs ~0 na regressão). 3. **Boas práticas de ML** foram aplicadas: pipeline de pré-processamento, divisão estratificada, semente fixa, baselines de comparação, métricas adequadas a cada tarefa e documentação.

Aviso importante (que está no próprio notebook): classificação e regressão são problemas diferentes, então **suas métricas não devem ser comparadas diretamente** (não dá para dizer "0,83 de acurácia é pior que 0,90 de R²" — são coisas distintas).

Frases-resumo para slide: - "Classificar = em que faixa o aluno está. Regredir = qual nota exata ele tira." - "O baseline existe para provar que a rede aprendeu, e não chutou." - "Padronizar os dados não é opcional em redes neurais — é o que torna o treino estável."